

```
In [ ]: ## MARKET BASKET ANALYSIS WITH PYTHON - AMAZON ##
# BY - AKRITI TRIPATHI
```

```
In [1]: # Basic Libraries
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# Settings
sns.set(style="whitegrid")
```

```
In [3]: import pandas as pd

df = pd.read_csv(r"C:\Users\tripa\Downloads\Amazon.csv")
df.head()
```

Out[3]:

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Recommendation_Frequency
0	2023/06/08 7:50:55 PM GMT+5:30	65	Prefer not to say	Less than once a month	Clothing and Fashion	Yes
1	2023/06/09 9:37:44 AM GMT+5:30	20	Male	Once a week	Groceries and Gourmet Food;Beauty and Personal...	No
2	2023/06/11 11:26:54 PM GMT+5:30	42	Male	Once a week	Groceries and Gourmet Food;Beauty and Personal...	Sometimes
3	2023/06/08 5:17:10 PM GMT+5:30	65	Others	Once a month	Beauty and Personal Care;Clothing and Fashion;...	No
4	2023/06/11 10:59:30 PM GMT+5:30	45	Female	Once a week	Beauty and Personal Care;Clothing and Fashion;...	Sometimes

5 rows × 7 columns

```
In [7]: ##TASK 1---(DATA CLEANING AND PREPARATION)
# Remove duplicates or inconsistent survey responses
df = df.drop_duplicates()
```

```
In [15]: # fix column name
df.columns = df.columns.str.strip().str.replace(" ", "_")
```

```
In [16]: # Handle Missing Values
df.isnull().sum()
```

```
Out[16]: Timestamp      0
age                    0
Gender                 0
Purchase_Frequency    0
Purchase_Categories    0
Personalized_Recommendation_Frequency  0
Browsing_Frequency    0
Product_Search_Method  0
Search_Result_Exploration  0
Customer_Reviews_Importance  0
Add_to_Cart_Browsing  0
Cart_Completion_Frequency  0
Cart_Abandonment_Factors  0
Saveforlater_Frequency  0
Review_Left           0
Review_Reliability    0
Review_Helpfulness    0
Personalized_Recommendation_Frequency  0
Recommendation_Helpfulness  0
Rating_Accuracy        0
Shopping_Satisfaction  0
Service_Appreciation   0
Improvement_Areas      0
transaction            0
dtype: int64
```

```
In [ ]: #Categorical → fill with mode
#Numerical → fill with mean/median
```

```
In [14]: df['Product_Search_Method'].fillna(df['Product_Search_Method'].mode()[0], inplace=True)
```

```
In [17]: # Standardize Categorical entries
df['Gender'] = df['Gender'].str.lower().replace({
    'male': 'Male',
    'female': 'Female'
})
```

```
In [ ]: ##"Male", "male", "M" should NOT be treated differently"##
```

```
In [18]: ##Convert numerical rating columns
cols = ['Shopping_Satisfaction', 'Customer_Reviews_Importance']

for col in cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
In [ ]: ##"80% of real-world data science is cleaning – if this step is weak, everything fails."#
```

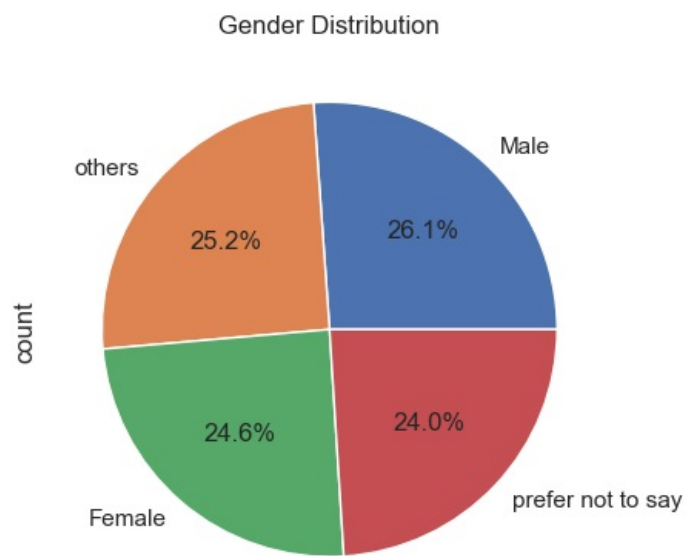
```
In [ ]: ##TASK 2- DESCRIPTIVE BEHAVIOR ANALYSIS#
```

```
In [20]: # Summarize customer demographics
df['age'].describe()
df['Gender'].value_counts()
```

```
Out[20]: Gender
Male                209
others              202
Female              197
prefer not to say   192
Name: count, dtype: int64
```

```
In [21]: #Visualization#
import matplotlib.pyplot as plt

df['Gender'].value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.title("Gender Distribution")
plt.show()
```



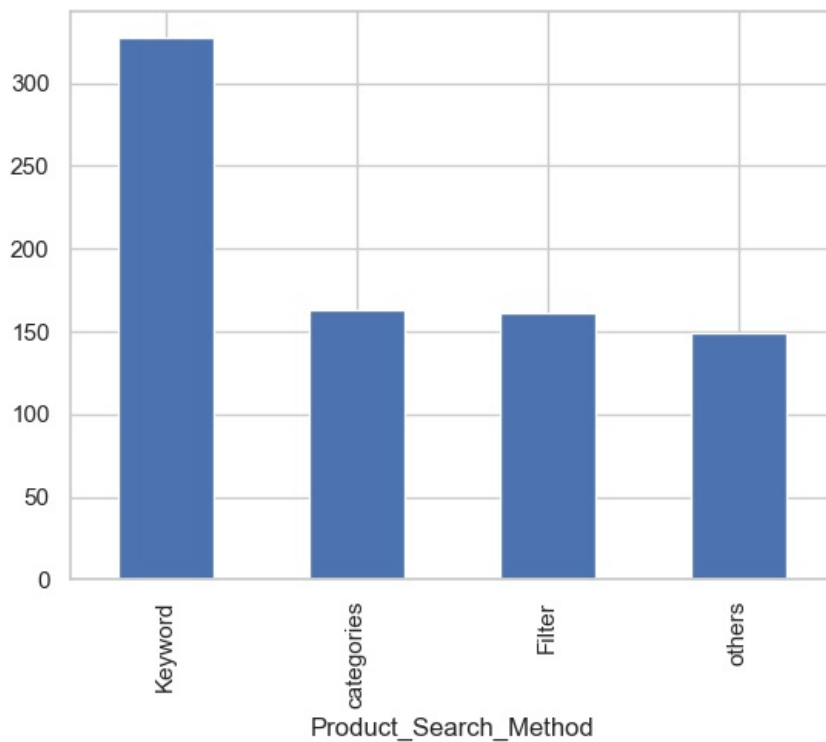
```
In [22]: # Analyze overall Purchase Frequency and most popular product categories.#
df['Purchase_Frequency'].value_counts()
```

```
Out[22]: Purchase_Frequency
Once a month          168
Once a week           159
Multiple times a week  159
Few times a month     158
Less than once a month 156
Name: count, dtype: int64
```

```
In [ ]: #Insight:
#High frequency = loyal users
#Low frequency = potential churn
```

```
In [24]: # Popular product Categories.
df['Product_Search_Method'].value_counts().plot(kind='bar')
```

```
Out[24]: <Axes: xlabel='Product_Search_Method'>
```



```
In [25]: # identify top browsing methods and most common cart abandonment factors.
df['Cart_Abandonment_Factors'].value_counts()
```

```
Out[25]: Cart_Abandonment_Factors
High shipping costs                224
Found a better price elsewhere     206
Changed my mind or no longer need the item 194
others                             176
Name: count, dtype: int64
```

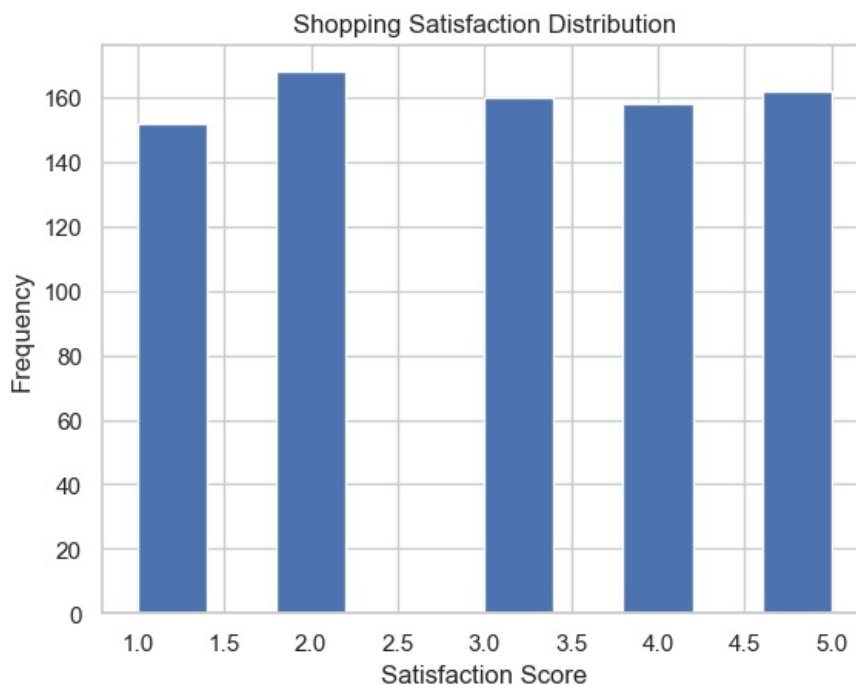
```
In [ ]: #Insight Example:
#High shipping cost → major issue
#Complex checkout → UX problem
```

```
In [26]: # Calculate mean and median satisfaction, recommendation helpfulness, and rating accuracy.
df[['Shopping_Satisfaction', 'Rating_Accuracy']].mean()
df[['Shopping_Satisfaction', 'Rating_Accuracy']].median()
```

```
Out[26]: Shopping_Satisfaction    3.0
Rating_Accuracy                 3.0
dtype: float64
```

```
In [28]: # generate summary statistics and visualization for key behavioral variables.
#Histogram (Distribution Understanding)
import matplotlib.pyplot as plt

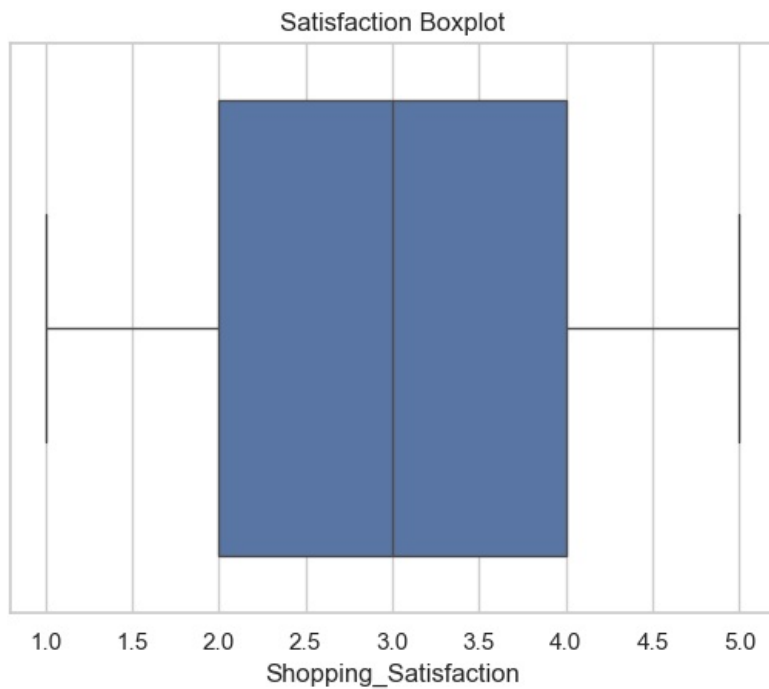
df['Shopping_Satisfaction'].plot(kind='hist', bins=10)
plt.title("Shopping Satisfaction Distribution")
plt.xlabel("Satisfaction Score")
plt.show()
```



```
In [ ]: #Insight:
#Left skew → unhappy customers
#Right skew → satisfied users
```

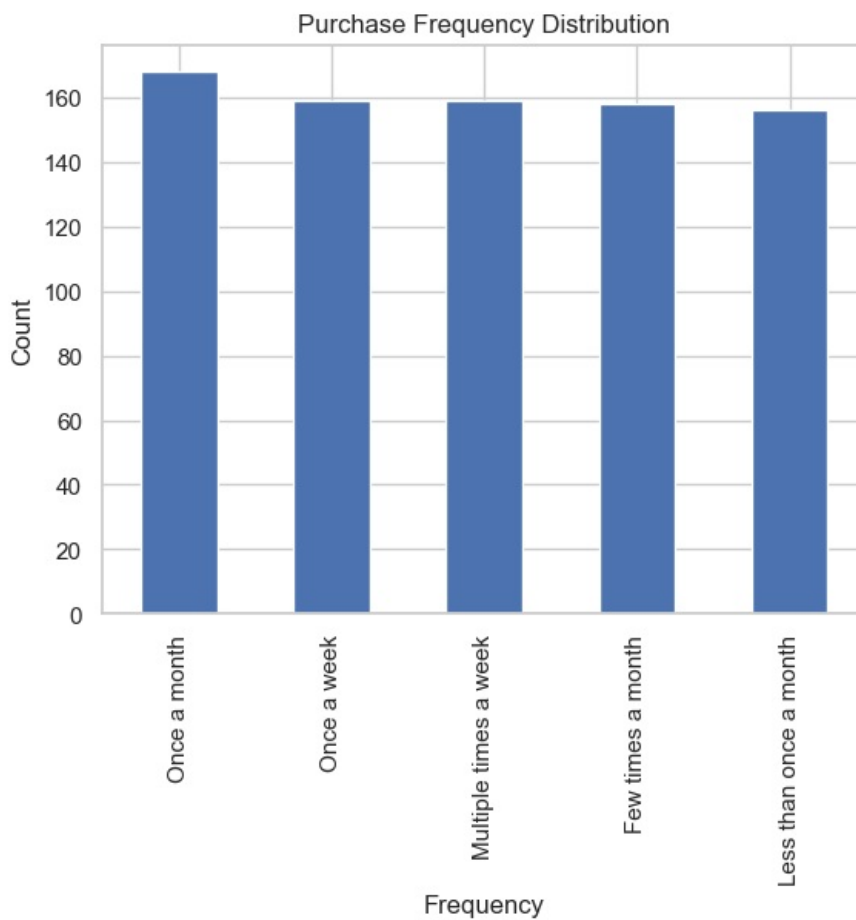
```
In [29]: #Box Plot (Outlier Detection)
import seaborn as sns

sns.boxplot(x=df['Shopping_Satisfaction'])
plt.title("Satisfaction Boxplot")
plt.show()
```

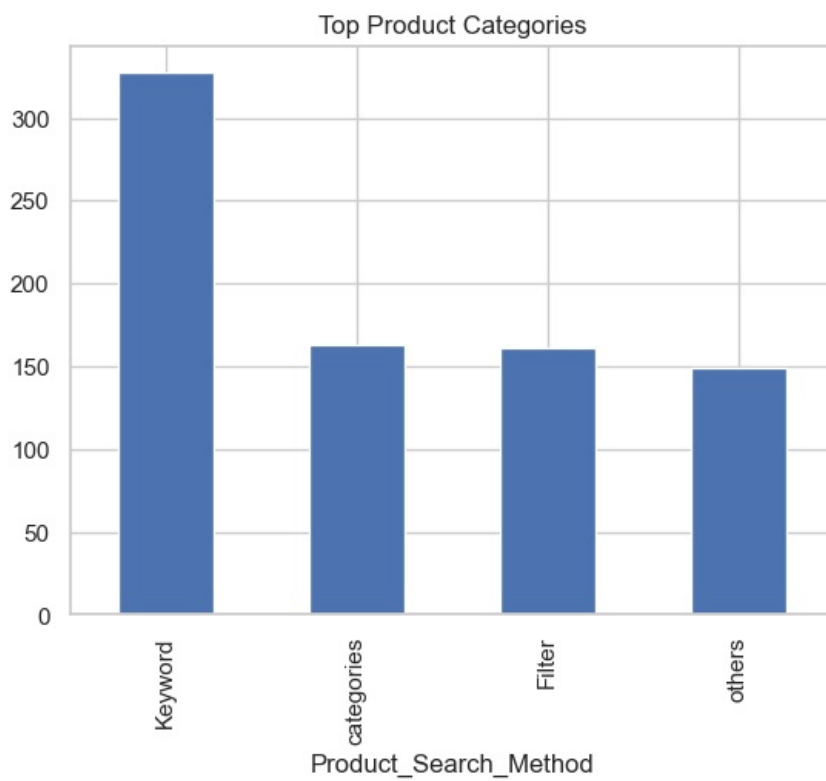


```
In [ ]: #insight
        #Outliers = extreme experiences (very good/bad)
```

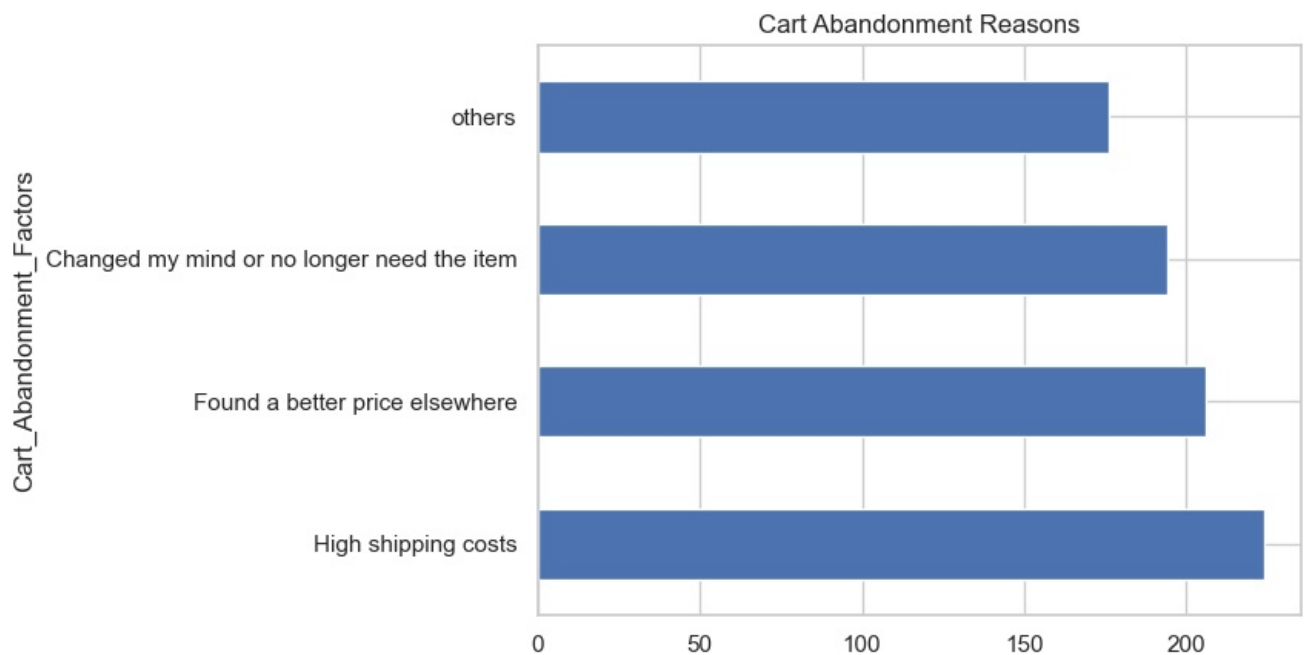
```
In [30]: #Bar Chart (Categorical Behavior)
df['Purchase_Frequency'].value_counts().plot(kind='bar')
plt.title("Purchase Frequency Distribution")
plt.xlabel("Frequency")
plt.ylabel("Count")
plt.show()
```



```
In [32]: #Top Product Categories
df['Product_Search_Method'].value_counts().head(10).plot(kind='bar')
plt.title("Top Product Categories")
plt.show()
```



```
In [33]: #Cart Abandonment Analysis
df['Cart_Abandonment_Factors'].value_counts().plot(kind='barh')
plt.title("Cart Abandonment Reasons")
plt.show()
```



```
In [34]: #Correlation Heatmap
sns.heatmap(df[cols].corr(), annot=True)
plt.title("Correlation Between Behavioral Variables")
plt.show()
```



```
In [ ]: #A positive correlation between recommendation helpfulness
#and satisfaction suggests that improving recommendation systems could directly enhance customer experience.
```

```
In [49]: ##TASK- 3 CUSTOMER SEGMENTATION AND PROFILIMG#
# segment customers based on purchase frequency and shopping satisfaction levels.
# Convert satisfaction to numeric
df['Shopping_Satisfaction'] = pd.to_numeric(df['Shopping_Satisfaction'], errors='coerce')

# Check unique values
df['Purchase_Frequency'].value_counts()
```

```
Out[49]: Purchase_Frequency
Once a month          168
Once a week           159
Multiple times a week  159
Few times a month     158
Less than once a month 156
Name: count, dtype: int64
```

```
In [50]: def segment(row):
    if row['Purchase_Frequency'] == 'High' and row['Shopping_Satisfaction'] >= 4:
        return "Frequent Buyers"

    elif row['Purchase_Frequency'] == 'Medium':
        return "Occasional Shoppers"

    else:
        return "At-Risk"

df['Segment'] = df.apply(segment, axis=1)
```

```
In [47]: #Analyze Each Segment.
df.groupby('Segment').mean(numeric_only=True)
```

```
Out[47]:
```

	age	Customer_Reviews_Importance	Personalized_Recommendation_Frequency	Rating_Accuracy	Shopping_Satisfaction
Segment					
At-Risk	35.17	3.05875	2.9525	2.96875	

```
In [54]: #Segment Size
df['Segment'].value_counts()
```

```
Out[54]: Segment
At-Risk      800
Name: count, dtype: int64
```

```
In [55]: #Demographics per Segment
df.groupby('Segment')['Gender'].value_counts()
```

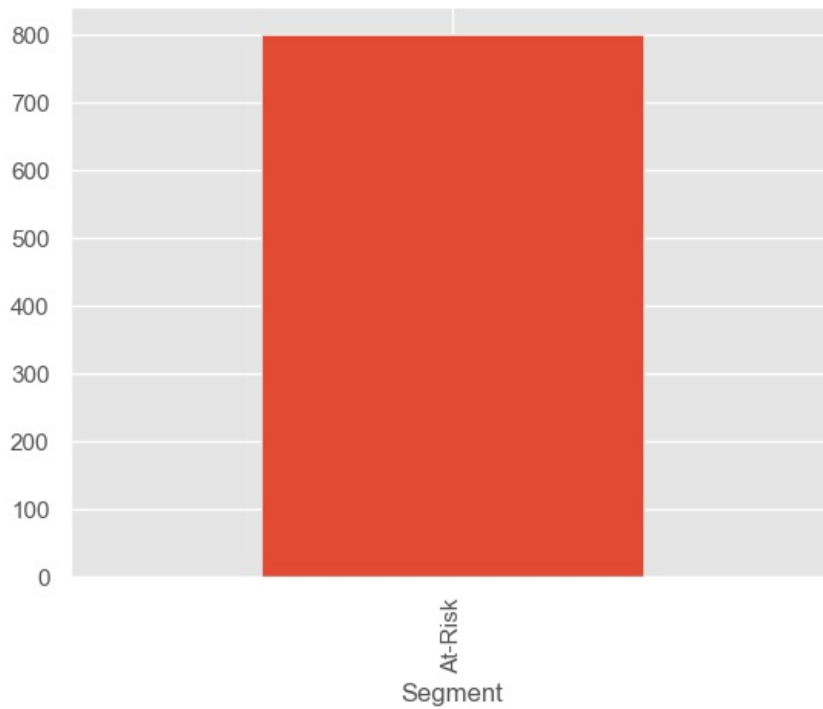
```
Out[55]: Segment Gender
At-Risk Male 209
others 202
Female 197
prefer not to say 192
Name: count, dtype: int64
```

```
In [56]: df.groupby('Segment')['age'].mean()
```

```
Out[56]: Segment
At-Risk 35.17
Name: age, dtype: float64
```

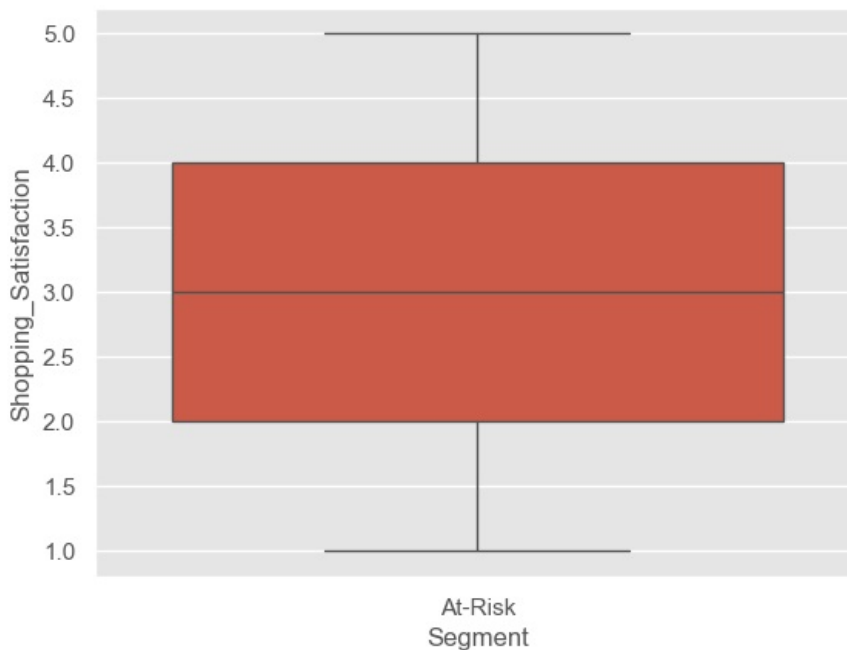
```
In [57]: #Visualize Segments
#Segment Distribution
df['Segment'].value_counts().plot(kind='bar')
```

```
Out[57]: <Axes: xlabel='Segment'>
```



```
In [58]: #Satisfaction by Segment
import seaborn as sns
sns.boxplot(x='Segment', y='Shopping_Satisfaction', data=df)
```

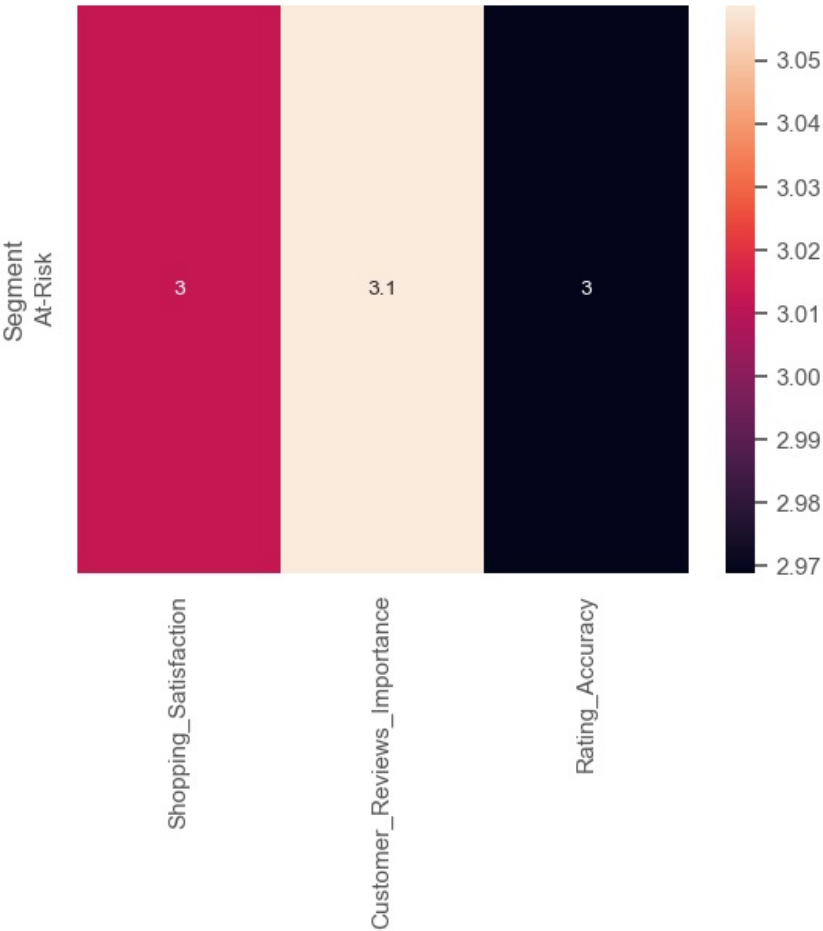
```
Out[58]: <Axes: xlabel='Segment', ylabel='Shopping_Satisfaction'>
```



```
In [65]: #Heatmap
sns.heatmap(df.groupby('Segment')[cols].mean(numeric_only=True), annot=True)
```



```
Out[65]: <Axes: ylabel='Segment'>
```



```
In [66]: #K-Means Clustering
#Select Features
features = df[['Shopping_Satisfaction', 'Customer_Reviews_Importance']].dropna()
```

```
In [71]: #apply k-means
df.groupby('Cluster')[['Shopping_Satisfaction', 'Customer_Reviews_Importance']].mean()
```

Out[71]:

	Shopping_Satisfaction	Customer_Reviews_Importance
Cluster		
0.0	4.212014	3.950530
1.0	2.922830	1.636656
2.0	1.500000	3.980583

```
In [72]: #Visualize Clusters
import matplotlib.pyplot as plt

plt.scatter(features['Shopping_Satisfaction'],
            features['Customer_Reviews_Importance'],
            c=df.loc[features.index, 'Cluster'])

plt.xlabel("Satisfaction")
plt.ylabel("Review Importance")
plt.title("Customer Clusters")
plt.show()
```



```
In [ ]: #Insight:
#Cluster 0 → high satisfaction (loyal)
#Cluster 1 → medium (convertible)
#Cluster 2 → low (risk)
#“Segmentation is where data turns into money – because now you know WHO to target and HOW.”
```

```
In [73]: ##TASK 4 RECOMMENDATION AND REVIEW INSIGHTS
# Examine the relationship between recommendation helpfulness and shopping satisfaction.
cols = ['Shopping_Satisfaction', 'Recommendation_Helpfulness',
        'Customer_Reviews_Importance', 'Rating_Accuracy']

for col in cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
In [74]: #Correlation Analysis
df[cols].corr()
```

```
Out[74]:
```

	Shopping_Satisfaction	Recommendation_Helpfulness	Customer_Reviews_Importance
Shopping_Satisfaction	1.000000	NaN	-0.03700
Recommendation_Helpfulness	NaN	NaN	NaN
Customer_Reviews_Importance	-0.037009	NaN	1.00000
Rating_Accuracy	-0.015644	NaN	0.03639

```
In [75]: #Visualization
import seaborn as sns
import matplotlib.pyplot as plt

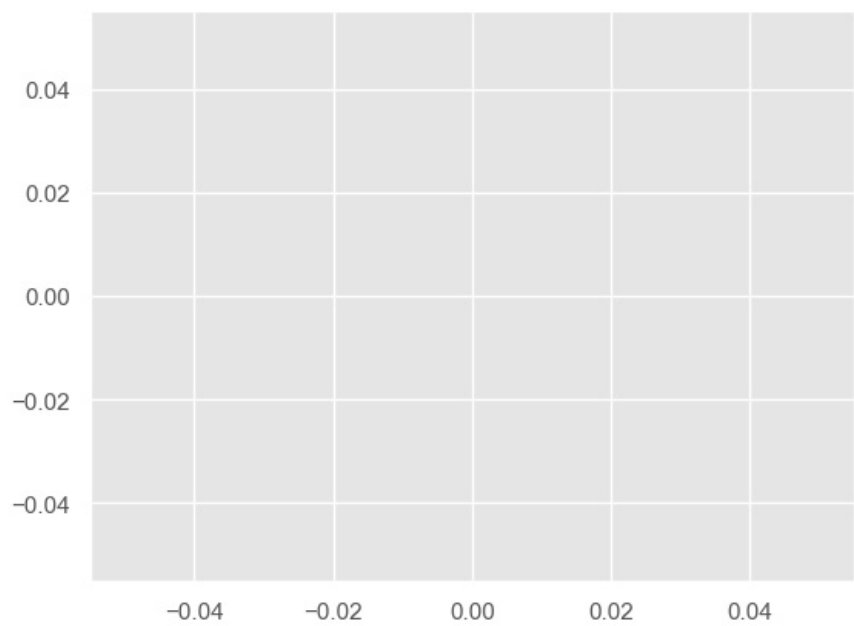
sns.heatmap(df[cols].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Between Reviews, Recommendations & Satisfaction")
plt.show()
```



```
In [ ]: #A strong positive correlation between recommendation helpfulness and satisfaction indicates that customers
#who find recommendations useful tend to have a better shopping experience.
```

```
In [76]: #Deep Relationship Analysis
#Recommendation vs Satisfaction
sns.scatterplot(
    x='Recommendation_Helpfulness',
    y='Shopping_Satisfaction',
    data=df
)
```

Out[76]: <Axes: >



```
In [ ]: #Insight:
#Upward trend → recommendations working
#No trend → system ineffective
```

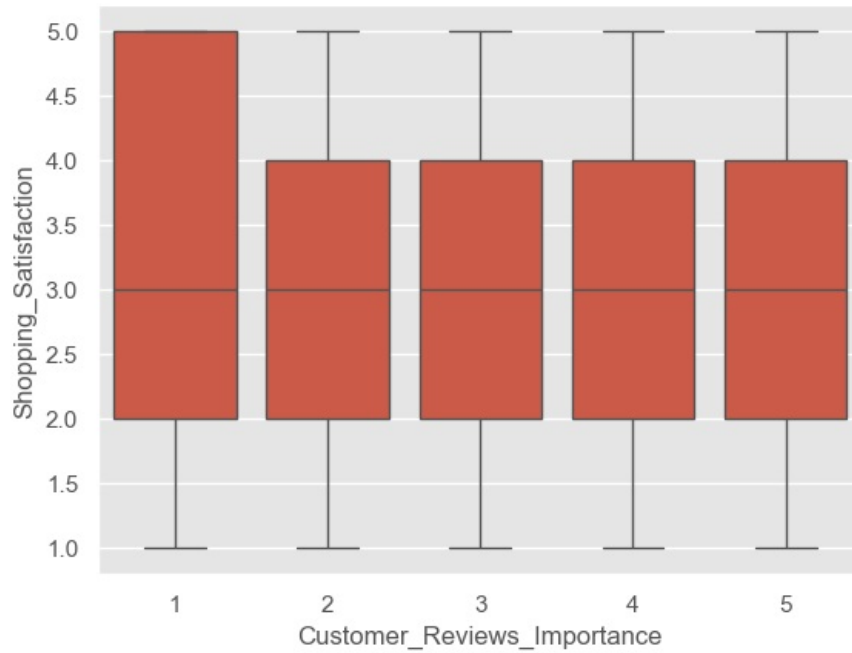
```
In [77]: #Reviews Importance vs Satisfaction
sns.boxplot(
```

```

x='Customer_Reviews_Importance',
y='Shopping_Satisfaction',
data=df
)

```

Out[77]: <Axes: xlabel='Customer_Reviews_Importance', ylabel='Shopping_Satisfaction'>



```

In [ ]: #Insight:
#Higher importance → higher satisfaction → reviews matter
#No change → reviews not impactful

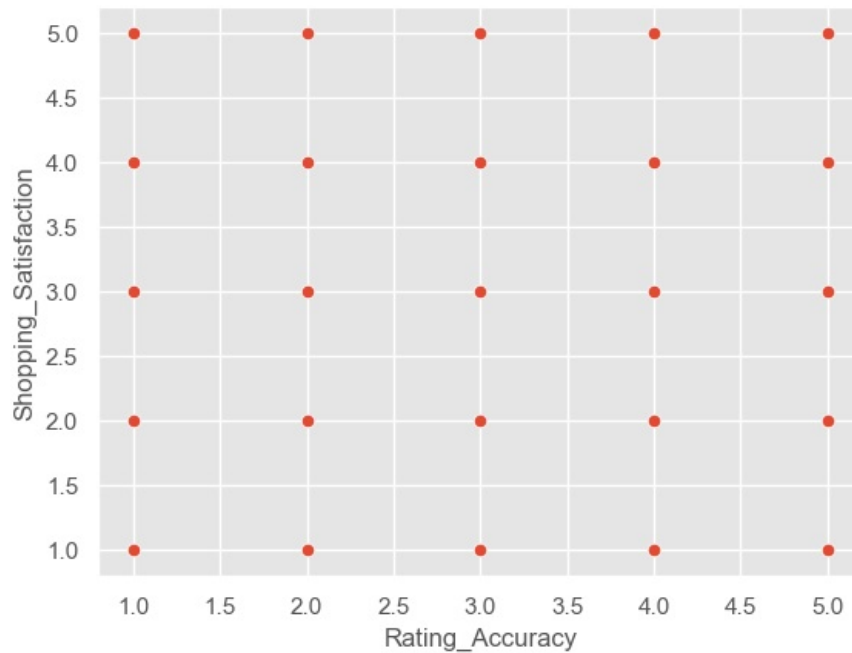
```

```

In [78]: #Rating Accuracy Impact
sns.scatterplot(
    x='Rating_Accuracy',
    y='Shopping_Satisfaction',
    data=df
)

```

Out[78]: <Axes: xlabel='Rating_Accuracy', ylabel='Shopping_Satisfaction'>



```

In [ ]: #Insight:
#Accurate ratings → trust → satisfaction ↑

```

```

In [88]: df['Recommendation_Helpfulness'].head()
df['Recommendation_Helpfulness'].isna().sum()

```

Out[88]: np.int64(800)

```

In [89]: #Customer Behavior Trends
#Check Engagement with Recommendations

```

```
df['Recommendation_Helpfulness'].value_counts()
```

```
Out[89]: Series([], Name: count, dtype: int64)
```

```
In [90]: #Visualization:
counts = df['Recommendation_Helpfulness'].value_counts()

if not counts.empty:
    counts.plot(kind='bar')
    plt.title("Recommendation Helpfulness Distribution")
else:
    print("No data available to plot.")
```

No data available to plot.

```
In [91]: #Combine Insights
df.groupby('Recommendation_Helpfulness')['Shopping_Satisfaction'].mean()
```

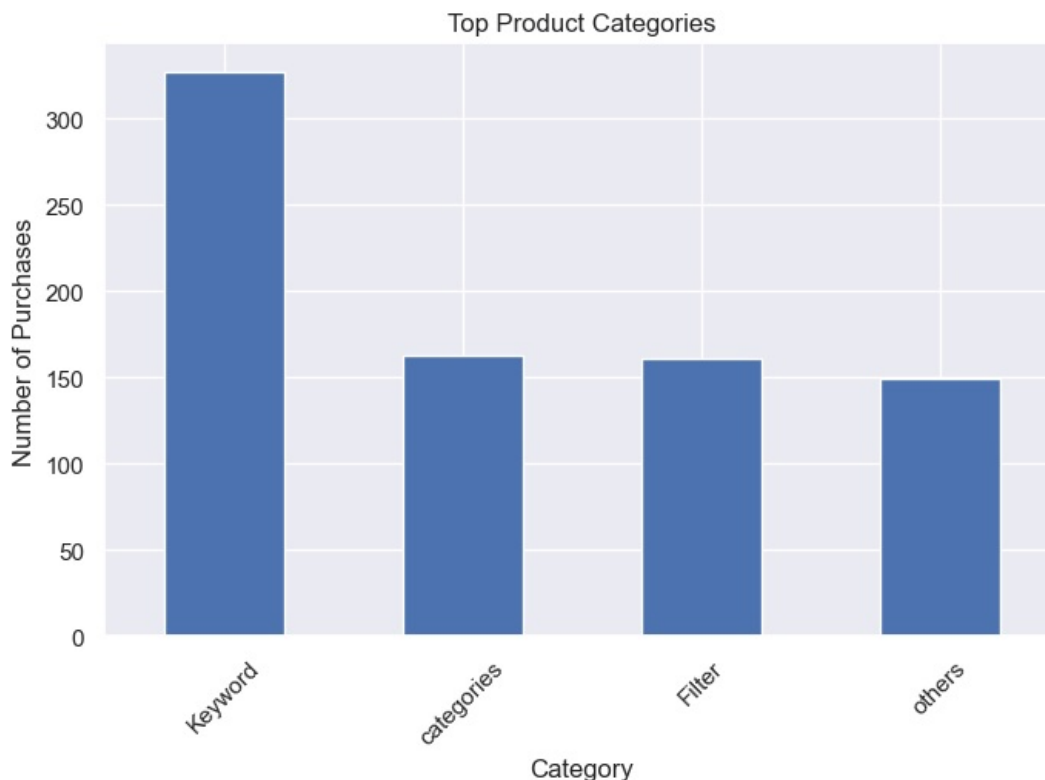
```
Out[91]: Series([], Name: Shopping_Satisfaction, dtype: float64)
```

```
In [ ]: #Customers who rate recommendations as highly helpful show significantly higher satisfaction levels,
#indicating a direct impact of recommendation quality on user experience.
```

```
In [93]: ##TASK- 5 VISUALIZATION AND REPORTING
#Create attractive visualizations
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use('ggplot')
sns.set(rc={'figure.figsize':(8,5)})
```

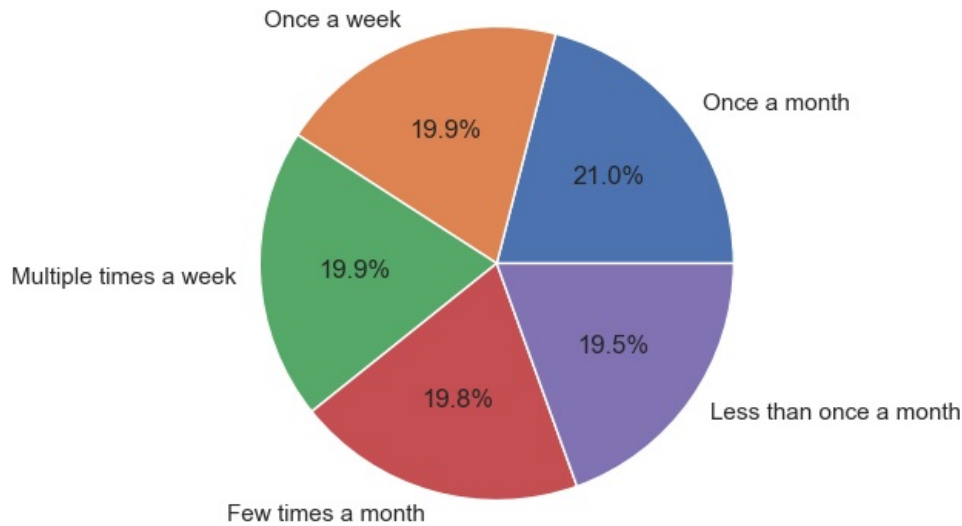
```
In [95]: #Purchase Categories Visualization
df['Product_Search_Method'].value_counts().plot(kind='bar')
plt.title("Top Product Categories")
plt.xlabel("Category")
plt.ylabel("Number of Purchases")
plt.xticks(rotation=45)
plt.show()
```



```
In [ ]: #Insight:
#Categories with highest frequency represent strong demand and cross-selling opportunities.
```

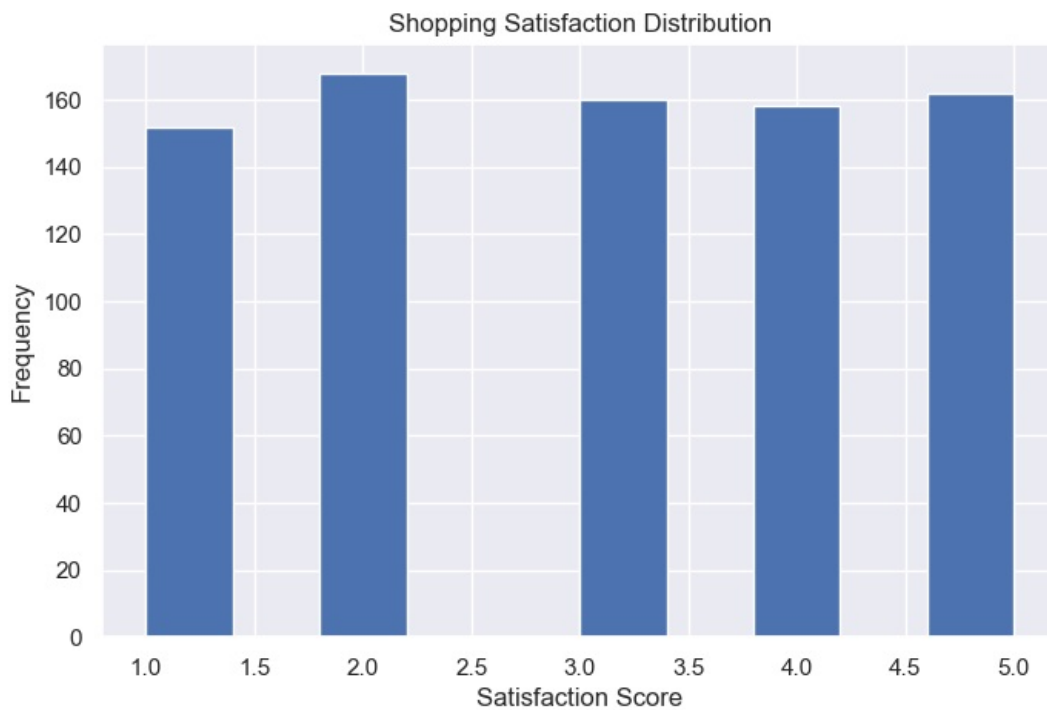
```
In [96]: #Browsing Frequency Distribution
df['Purchase_Frequency'].value_counts().plot(kind='pie', autopct='%1.1f%%')
plt.title("Purchase Frequency Distribution")
plt.ylabel("")
plt.show()
```

Purchase Frequency Distribution



```
In [ ]: #Insight:
#Majority users are occasional buyers → scope to increase engagement.
```

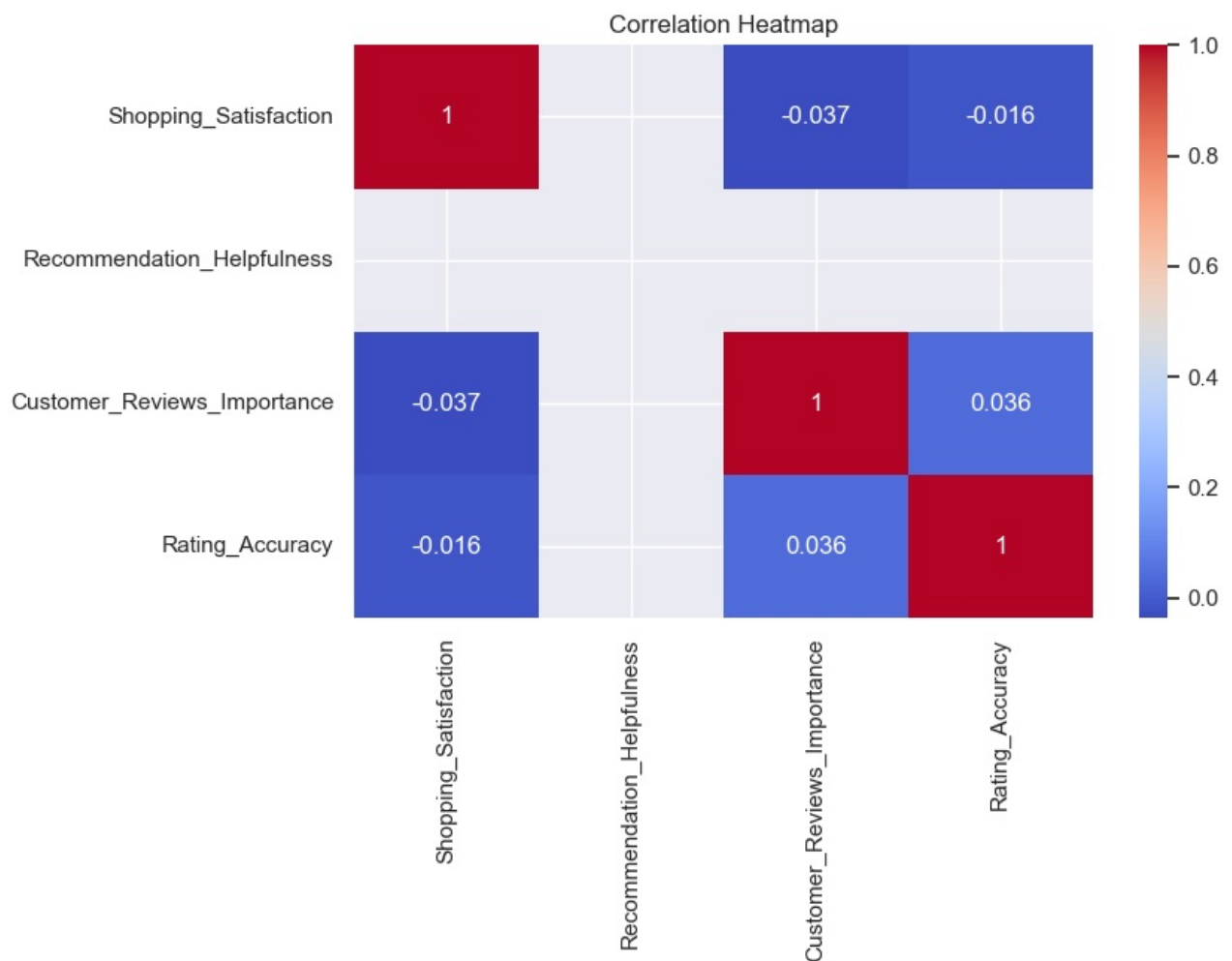
```
In [97]: #Satisfaction Levels Distribution
df['Shopping_Satisfaction'].plot(kind='hist', bins=10)
plt.title("Shopping Satisfaction Distribution")
plt.xlabel("Satisfaction Score")
plt.show()
```



```
In [ ]: #Insight:
#Right skew → satisfied users
#Left skew → dissatisfaction risk
```

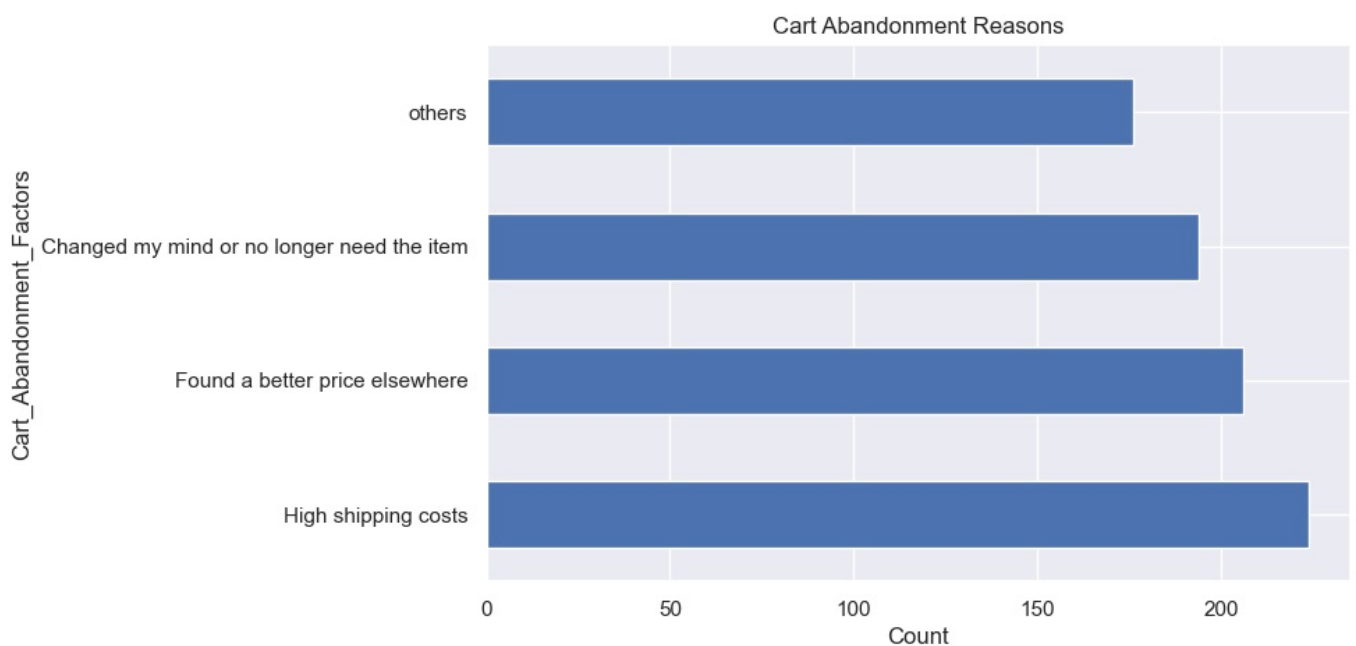
```
In [98]: #Correlation Heatmap
cols = ['Shopping_Satisfaction', 'Recommendation_Helpfulness',
        'Customer_Reviews_Importance', 'Rating_Accuracy']

sns.heatmap(df[cols].corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



```
In [ ]: #Insight:
#Strong correlations indicate key drivers of customer satisfaction.
```

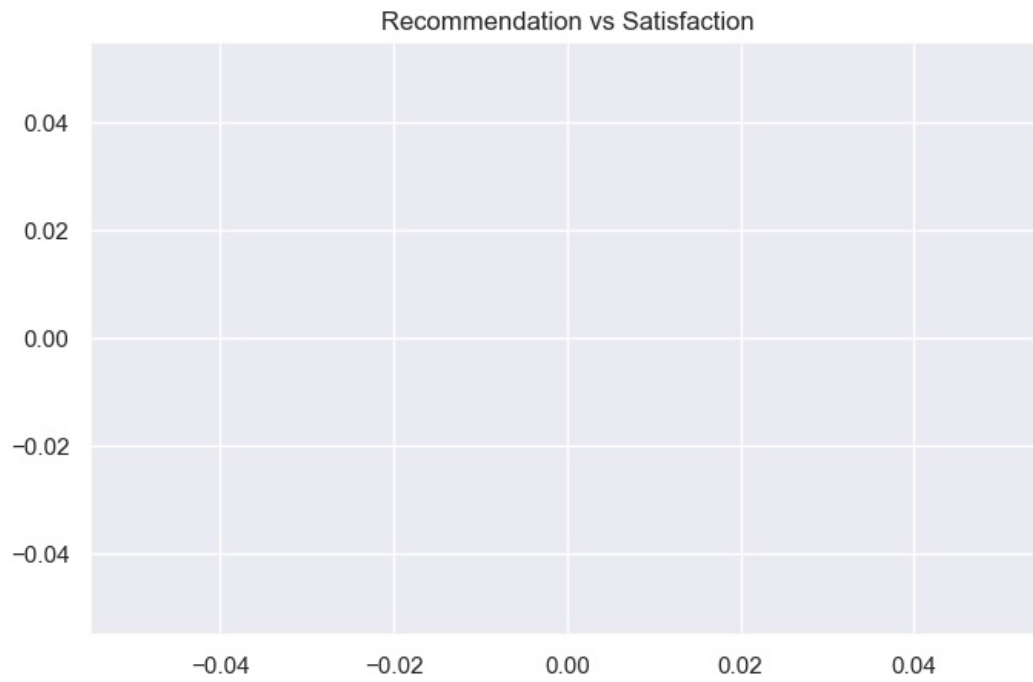
```
In [99]: #Cart Abandonment Analysis
df['Cart_Abandonment_Factors'].value_counts().plot(kind='barh')
plt.title("Cart Abandonment Reasons")
plt.xlabel("Count")
plt.show()
```



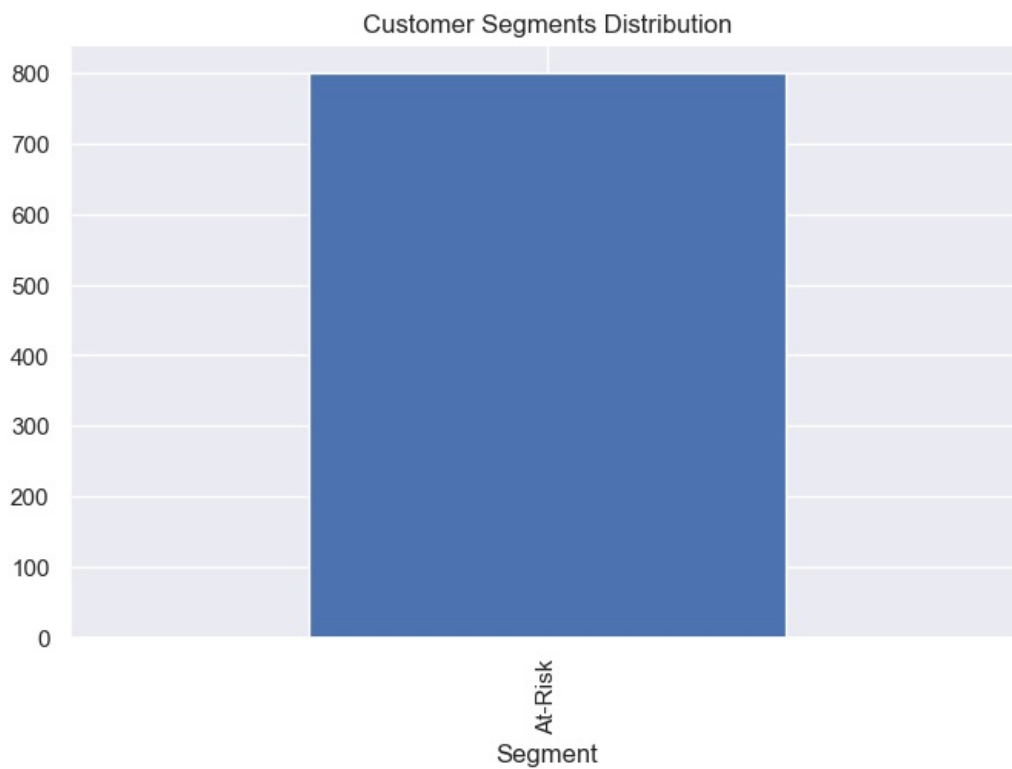
```
In [ ]: #Insight:
#Helps improve checkout process and reduce revenue loss.
```

```
In [100]: #Recommendation vs Satisfaction
sns.scatterplot(
    x='Recommendation_Helpfulness',
    y='Shopping_Satisfaction',
    data=df)
```

```
)
plt.title("Recommendation vs Satisfaction")
plt.show()
```



```
In [101]: #Segment Visualization
df['Segment'].value_counts().plot(kind='bar')
plt.title("Customer Segments Distribution")
plt.show()
```



```
In [ ]: #A significant portion of customers fall into the "Occasional Shoppers" segment, presenting an opportunity for
```